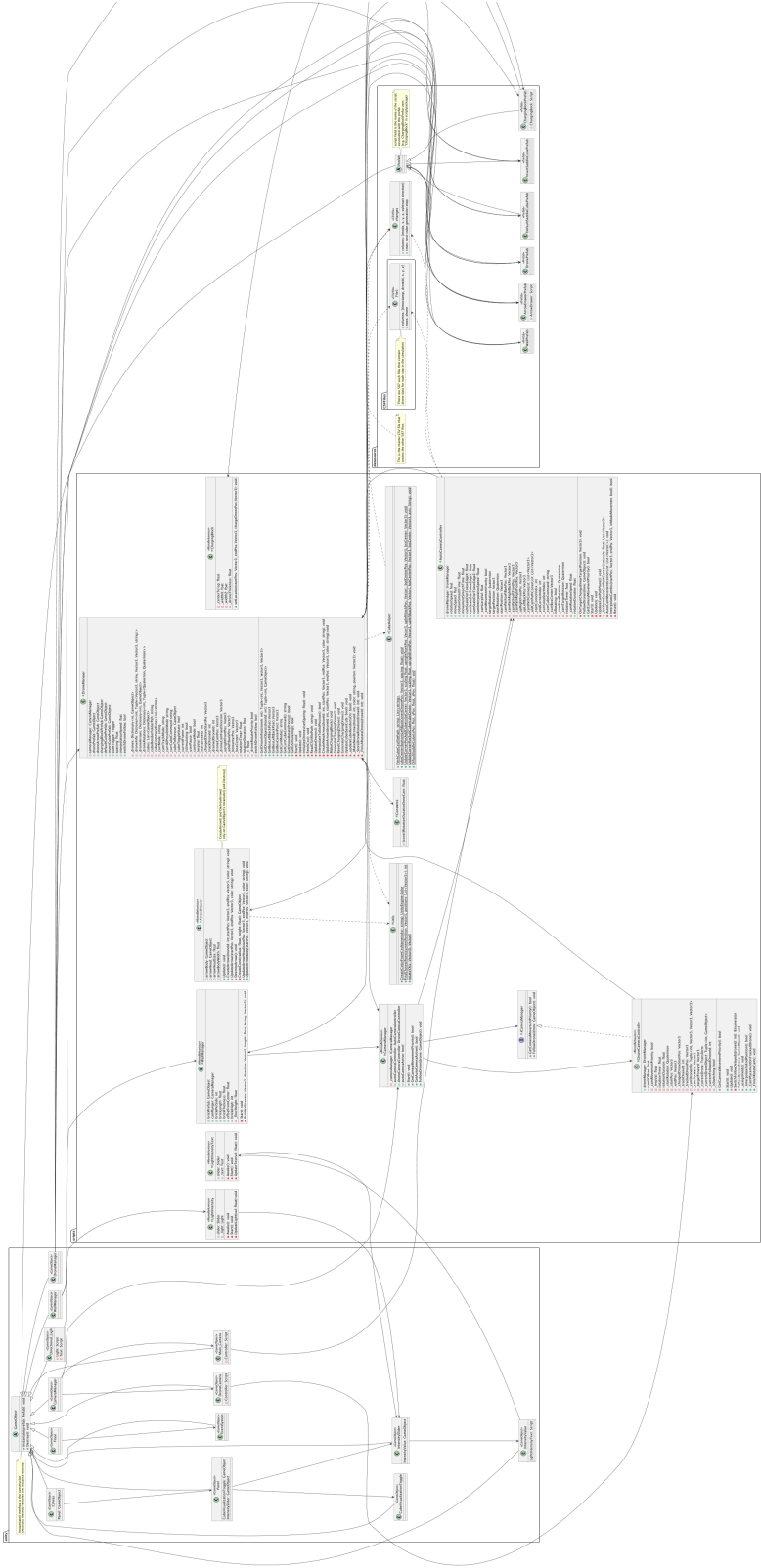
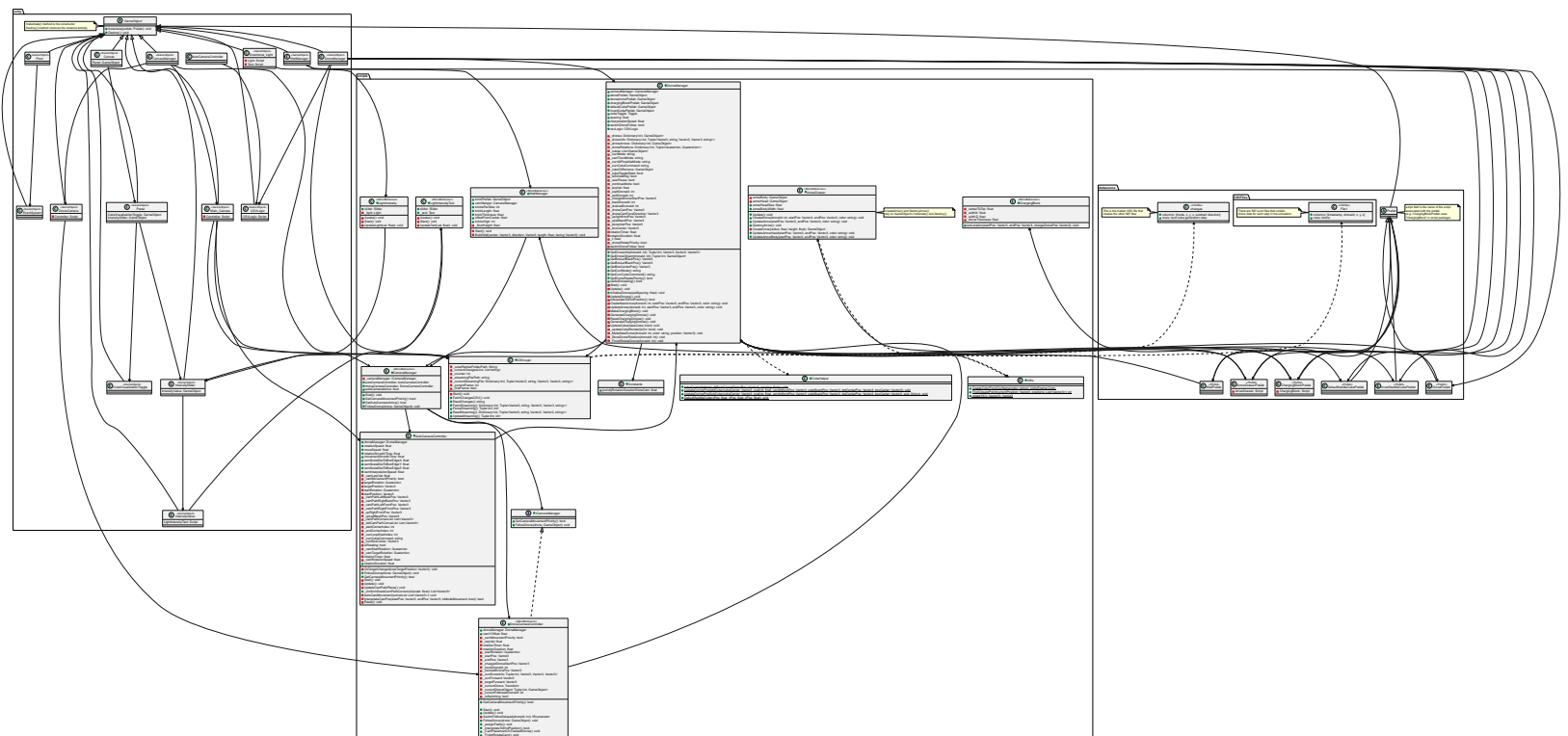


Initial Project Design:



Refactored Project Design:

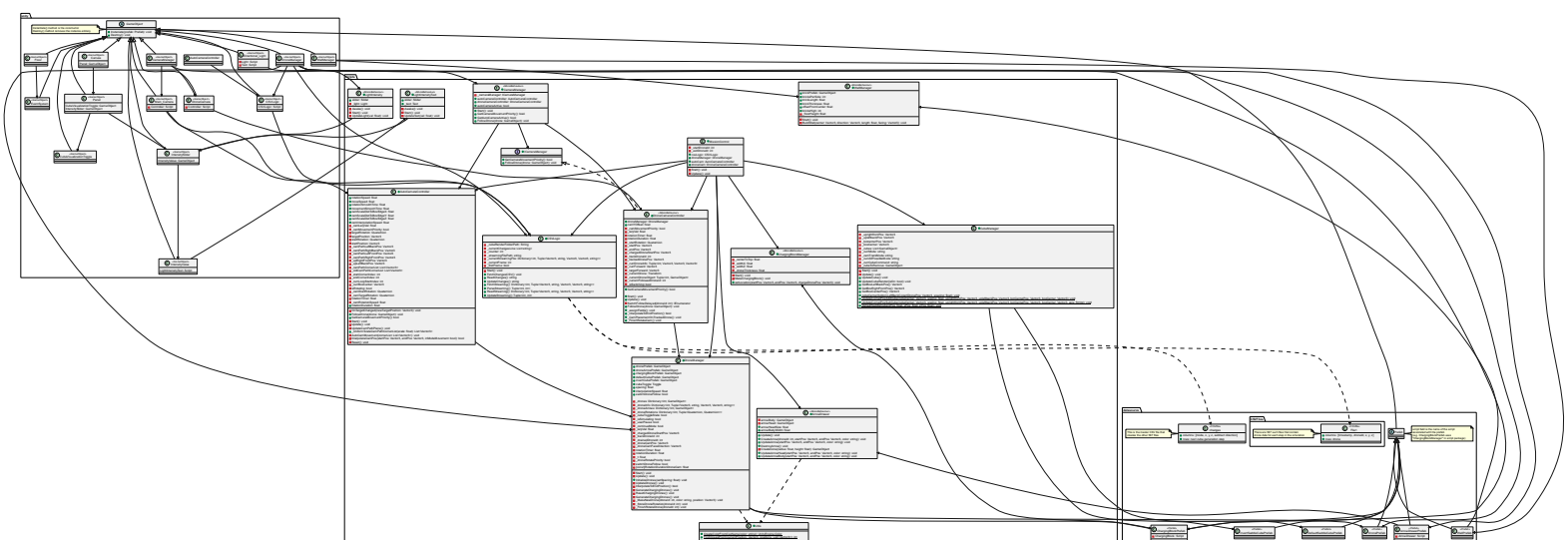


List of Major Changes:

The major change in design is that we added the csvLogic class to the codebase. As a result, we also removed functions from DroneManager (FetchCSV() and ReadCSV()) and CubeHelper (FetchCubeCSV()). We were also able to remove some fields that were used by both classes. Finally, now only csvLogic has a dependency relationship with the two files.

FetchChangesCSV(), ReadChanges(), and UpdateChanges() open the changes.csv file, reads the lines into a private field, and returns a line at a time through either Read() or Update(). The FetchStreaming() function opens the first drone movement file, ParseStreaming() parses the file and populates a drone information dictionary, ReadStreaming() returns the dictionary, and UpdateStreaming() repeats the process for the next drone movement file.

Redesign:



List of Major Changes:

First, we added the class `MissionControl` and gave it an instance of all other relevant classes (`csvLogic`, `CubeManager`, `DroneManager`, etc.). `csvLogic` is responsible for parsing and streaming CSV data, `DroneManager` is responsible solely for managing drones, `CubeManager` is responsible for managing cubes, while `CameraManager`, `AutoCameraController`, and `DroneCameraController` are responsible for the camera algorithms. With instances of all of these classes, `MissionControl` can now update each class in the correct order in `Update()` frame by frame.

Second, we created a `CubeManager` class to handle all logic related to the cubes, and combined the static methods in `CubeHelper`. This created a direct class that works with `MissionControl` to manage all cube instantiation, updates, and deletions.

Third, we debloated `DroneManager`. We removed methods and fields that handled logic to manage cubes, draw and update arrows, and getters that were required for other classes that depended on `DroneManager`.

Finally, we reduced ties from specific classes to `DroneManager` and instead directed them to `MissionControl`. `AutoCameraControl.cs` previously needed an instance of `DroneManager` to access a field and two methods, which are now communicated through `MissionControl`. There are similar adjustments for `DroneCameraController`. `DroneManager` also does not require instances of `CubeManager` (previously `CubeHelper`) as cube logic is handled through `MissionControl`.

Reflection of Successful Refactor

At the time of submitting Milestone 2, we decided to refactor the system piece by piece and update the flow of the UCs, rather than update the codebase by the UCs. Our plan was to debloat DroneManager.cs to separate the multiple responsibilities this class manages.

In the refactored diagram, there is another class called CSVLogic.cs that is used by directly by DroneManager and indirectly by AutoCameraController and DroneCameraController. We successfully implemented this refactor.

Successes

We were able to successfully limit the number of classes that read from the csv files. Now, only the csvLogic class is responsible for managing the files. Due to this refactoring, we were able to couple the logic that parses both files into one class. Previously, the class CubeHelper was responsible for parsing the changes.csv file that informs the Cube behavior. DroneManager also contained two functions, FetchCSV() and ReadCSV(), that were responsible for parsing and populating a dictionary containing drone information. These functionalities are now handled in csvLogic (the functionality details are on page TK).

Challenges

When refactoring DroneManager to only rely on csvLogic, we realized that the previous functions FetchCSV() and ReadCSV() also contained more business logic that had to be moved. This logic usually involved setting flags that are used to update the simulation in the correct order. By moving the parsing logic to a separate class, we caused continuity bugs as files were calling updates on pieces that were not supposed to update, or were already updated by a different part of the simulation.

Update Mode

First, please find a snippet of ReadCSV() below:

```
if (_currMode == null || (mode != _currMode && mode != "secondary"))
{
    UpdateCube();
    Debug.Log("enter" + mode);
    _currMode = mode;
}
// Debug.Log(_currAllPossibleMode);
if (_currAllPossibleMode == null || mode != _currAllPossibleMode)
{
    _currAllPossibleMode = mode;
    // Debug.Log(_currAllPossibleMode);
}
```

Earlier in ReadCSV(), the current drone movement was parsed and populated into a drone information dictionary. While the file was parsing, we updated the variable "mode", and then used it in this snippet to determine whether a cube needed to be updated in the simulation by calling UpdateCube(). Once we implemented csvLogic() and removed FetchCSV() and ReadCSV(),

we no longer had a way to manage the local mode variable without looping through the dictionary a second time or coupling the update logic back into the parse, which defeats the purpose of the separation. We solved this bug by placing it into DroneManager's Update() function, with added logic to ensure UpdateCube() is called at the right time.

In csvLogic, we return a tuple with _startDroneId and _exitDroneId to keep track of which drone is exiting the cube:

```
mode = values[11];

Vector3 startPos = Utils.rotateYZ(new Vector3(start_x, start_y, start_z));
Vector3 endPos = Vector3.zero;
// the condition indicates dying drone exiting the cube
if (float.Parse(values[10]) == -1.0f)
{
    // Debug.Log(droneId);
    _exitDroneId = droneId;
    //endPos = _chargedDroneStartPos;
    //// store the facing direction of exit drone
    //_exitDroneFaceDirection = (endPos - startPos).normalized;
    //_exitDroneFaceDirection.y = 0.0f; // don't want rotation on y axis
}
else
{
    endPos = Utils.rotateYZ(new Vector3(end_x, end_y, end_z));
}
_currentStreamingFile.Add(droneId, new Tuple<Vector3, string, Vector3, Vector3, string>
    (currPos, color, startPos, endPos, mode));
}
}
return (_startDroneId, _exitDroneId);
```

Then, we first check the mode in InitializeDrones() when the simulation first begins:

```
if (_currMode == null || (mode != _currMode && mode != "secondary"))
{
    UpdateCube();
    Debug.Log("enter" + mode);
    _currMode = mode;
}
// Debug.Log(_currAllPossibleMode);
if (_currAllPossibleMode == null || mode != _currAllPossibleMode)
{
    _currAllPossibleMode = mode;
    // Debug.Log(_currAllPossibleMode);
}
/// NEW STUFF END ///
GenerateChargingDrones();
UpdateDrones();
MakeChargingBlock();
```

Additionally, in DroneManager.Update(), we included much more logic to unpack the two Id numbers, check that they represent a drone that is exiting the current cube, create an exiting drone, collect the current mode manually, and update both the drones and cubes in GenerateChargingDrones(), ResetChargingDrones(), UpdateDrones(), and UpdateCube().

```
_lerpVal = 0.0f;
var result = csvLogic.UpdateStreaming();

// assign the drained drone end position for interpolation to go to charging block
_droneInfo = csvLogic.ReadStreaming();
// csvLogic has default -2 if no update for a new starting drone
if (result.Item1 != -2)
{
    _startDroneId = result.Item1;
}
// csvLogic has default -2 if no drone is exiting for this CSV
if (result.Item2 != -2)
{
    Debug.Log(_exitDroneId);
    _exitDroneId = result.Item2;
    _droneInfo[_exitDroneId] = new Tuple<Vector3, string, Vector3, Vector3,
        string>(_droneInfo[_exitDroneId].Item1, _droneInfo[_exitDroneId].Item2,
        _droneInfo[_exitDroneId].Item3, _chargedDroneStartPos, _droneInfo[_exitDroneId].Item5);
}
// get a key for the current mode
string mode = "run";
int i = 0;
foreach (var key in _droneInfo.Keys)
{...}
GenerateChargingDrones(); // add in the 997 and 998
UpdateDrones();
if (_currMode == null || (mode != _currMode && mode != "secondary"))
{
    UpdateCube();

    _currMode = mode;
}

if (_currAllPossibleMode == null || mode != _currAllPossibleMode)
{
    _currAllPossibleMode = mode;
    // Debug.Log(_currAllPossibleMode);
}
_StoreDroneRotation(_trackDroneId);
ResetChargingDrones();
```

Drone Movement

Another set of logic that was displaced from ReadCSV() was when and how to update the drone movements. Previously, the simulation would begin, and call Start() for each game object. In DroneManager, Start() called InitializeDrones() to begin the logic.

InitializeDrones() previously only called FetchCSV() and MakeChargingBlock(), which would parse and render the drones, and render the charging block respectively. FetchCSV() would call UpdateDrones(), and ReadCSV() would call GenerateChargingDrones() and UpdateCube(). After removing FetchCSV(), we now rely on the csvLogic class:

```
public void InitializeDrones(float setSpacing)
{
    spacing = setSpacing;
    _droneInfo = new Dictionary<int, Tuple<Vector3, string, Vector3, Vector3, string>>(csvLogic.FetchStreaming());
    _droneInfo = csvLogic.ReadStreaming();
    string mode = "run";
    if (_currMode == null || (mode != _currMode && mode != "secondary"))
    {
        UpdateCube();
        Debug.Log("enter" + mode);
        _currMode = mode;
    }
    // Debug.Log(_currAllPossibleMode);
    if (_currAllPossibleMode == null || mode != _currAllPossibleMode)
    {
        _currAllPossibleMode = mode;
        // Debug.Log(_currAllPossibleMode);
    }
    GenerateChargingDrones();
    UpdateDrones();
    MakeChargingBlock();
}
```

In the refactored code, InitializeDrones() receives the parsed data, performs the new logic on updating cube rendering, and updates the drones through GenerateChargingDrones() and UpdateDrones(). Every frame, DroneManager.Update() (previous page) now similarly calls GenerateChargingDrones(), ResetChargingDrones(), UpdateDrones(), and UpdateCube().

Determining the correct order to call these functions was challenging.

GenerateChargingDrones() must be executed before calling UpdateDrones(). This is because GenerateChargingDrones() manually produces special drones that leave cube and return to the charging block with ID 997, while UpdateDrones() removes them from the render. The gameObject drone 997 would exist inside of the dictionary containing drone information, but would not exist inside of the game object drones that were rendered on the screen, so the mismatch would cause a dictionary key error. This was not obvious, as the execution order was implicit in the previous design.

Similarly, calling UpdateCube() required first extracting the current mode from the CSV data and comparing it to both _currAllPossibleMode and _currMode. While _currAllPossibleMode was

used to continuously track the current mode from the data stream, `_currMode` was used to determine whether a meaningful mode transition had occurred. Only if the new mode differed from `_currMode` (and wasn't a transitional value like "secondary"), would `UpdateCube()` be triggered to reflect the change in simulation state.

Automatic Camera Movement

The automatic camera follows a distinct algorithm; it circles the current simulation then stays in a fixed position while the drones populate on screen for each iteration of drone movements. As the number of drones and cubes on screen increases, the camera must move back and up to keep them in view. To continually consider the positioning of the drones and cubes, `AutoCameraControl.cs` relied on `DroneManager.GetCurrCubeCommand()`, which is a getter that returns `DroneManager._currCubeCommand`.

`_currCubeCommand` is a field that contains the current line from `changes.csv`. We implemented the changes to `AutoCameraControl.cs` to rely only on `csvLogic` for csv data. However, we bumped into more continuity bugs. `csvLogic.ReadChanges()` is the function designed to take over `_currCubeCommand`. However, `DroneManager` is responsible for calling `csvLogic.UpdateChanges()` when the current line is finished. So, both `AutoCameraControl.cs` and `DroneManager()` would call `csvLogic.ReadChanges()` and `csvLogic.UpdateChanges()`, as they both read a line and update the counter. However, this would result in the counter skipping a line, as both classes updated in the same frame.

To solve this bug, we kept `DroneManager._currCubeCommand` and updated it through `csvLogic.ReadChanges()`, so `AutoCameraControl` can access the current cube command without overriding the current line. Unfortunately, this did not successfully decouple `AutoCameraControl` from `DroneManager` in terms of receiving csv data. However, it's necessary given that `DroneManager` must update the command via `csvLogic.UpdateChanges()`, and that this would overwrite `AutoCameraControl`'s calls to `csvLogic.ReadChanges()`.

```
void Start()
{
    dronePrefab.GetComponentInChildren<MeshRenderer>().enabled = false; // don't render the foundational drone
    chargingBlockPrefab.GetComponentInChildren<MeshRenderer>().enabled =
        false; // don't render the foundational charging block\
    defaultCubePrefab.GetComponentInChildren<MeshRenderer>().enabled = false; // don't render the foundational cube
    invertCubePrefab.GetComponentInChildren<MeshRenderer>().enabled =
        false; // don't render the foundational invert cube

    // NEW STUFF
    csvLogic.FetchChangesCSV(); // does same stuff as _cubeCommandLines =
    // CubeHelper.FetchCubeCsv(Path.Combine(Application.dataPath,
    //     _cubeRenderFolderPath)); in Start()
    _currCubeCommand = csvLogic.ReadChanges();|
```

Drone Camera Movement

The drone camera follows a single drone as it moves through the simulation. As this camera begins in the charging position, we relied on DroneManager to return the coordinate information on the charging drone (ID 997) to instantiate the original position of the drone. Once we had the position, we also collected its gameObject equivalent:

```
void Start()
{
    // Debug.Log(droneManager.GetDroneInfo(997));
    _currDroneInfo = droneManager.GetDroneInfo(997);
    // starts from the charged drone so only update this once

    _chargedDroneStartPos = _currDroneInfo.Item2;
    _currentDroneObject = droneManager.GetDroneObject(_currDroneInfo.Item1);
    FollowDrone(_currentDroneObject.Item2);
}
```

However, while we were debugging the appearance of charging drones in the first bug, this bug also occurred concurrently. First, GenerateChargingDrones() did not populate the charging drone upon Start(), as we moved the first instance of this function call into Update(). Then, when we realized the simulation was not rendering because DroneCameraController.cs needed a charging drone upon its Start() function, we moved the function call to occur first in InstantiateDrones() in addition to Update(). Then, the gameObject drone instance returned null, because the UpdateDrones() function was being called and deleting the drone before the camera could follow its position. So, we moved the function call of UpdateDrones() to Update() after we populated the cube line in DroneManager() to preserve its existence in the simulation and inform the camera.

Conclusion

The combination of bugs present in this refactor was challenging to solve. To completely refactor the code to reflect our full redesign, our team would more time to ensure that the simulation classes have the correct information in each frame by studying and adjusting the rate at which information is updated according to the execution chain. This challenge is within context of the next section.

Architectural Design Limitations

Refactoring a Unity-based simulation to decouple and debloat classes is easier said than done. Unity's engine is fundamentally frame-drive, so all visual updates, calculations, and script executions occur frame-by-frame. The CSV files that form the basis of the simulation are produced through an excel script, and cannot be altered or presented in an easier format. Thus, all business logic must be ingested and calculated in the correct order as the simulation is rendering in real time. Otherwise, bugs like the above will be met with each decoupling.

Initially, the parsing of CSV data was tightly integrated into DroneManager's ReadCSV() and FetchCSV(). These methods performed both data ingestion and simulation logic—updating drone positions, triggering camera movement, and adjusting visual elements like cubes based on mode. While this coupling worked during development, it violated clean design principles.

Although we were able to refactor the CSV data logic to a separate class with a functioning simulation, we were only able to refactor it while keeping DroneManager as a bloated class that is used by almost every other class. To completely break apart DroneManager to be solely responsible for managing drones, there are two options.

Mission Control

First, we can implement a centralized Mission Control class to follow a centralized controller pattern. This class would be responsible for pulling data from csvLogic and broadcasting the relevant information to all other classes at the correct time, and updating the information in and only if it is appropriate to do so.

This approach's major strength is that all simulation updates are made in a consistent, frame-by-frame order that mirrors Unity's execution order. This ensures that the logic to update the simulation is clean and traceable. It would also promote decoupling with the csv files, as csvLogic can still be the sole handler of the data files.

However, mission control requires tight coupling to completely control the internal behavior and timing requirements of every component it updates manually. In the future, as more camera algorithms and drone movement algorithms are added, Mission Control could potentially become a monolithic, bloated class itself.

Subscriber

The second approach is to implement the subscriber model, where csvLogic acts as a publisher that holds parsed data and notifies any interested component when new data is available. Each subscriber, such as DroneManager, would register with csvLogic and react to new data as it arrives.

This approach has loose coupling as components are modular and only respond to their own relevant data. It's also possible for new systems to subscribe to csvLogic without modifying existing code.

However, multiple subscribers may interpret the same data differently or miss updates due to timing alignments, as seen in previous bugs. This can cause a very complex debugging process to implement and maintain the design.